

LEARN BUILD SHARE

Practical Git & Version Control Guide (CLI, Git Bash, VS Code, PyCharm)

TABLE OF CONTENTS / QUICK INDEX

01	What is Git? (Beginner Analogies & Core Concepts)	Page 1
02	Getting a Git Server & Creating a Remote Repository	Page 1
03	Windows Setup: Git Bash Installation & Configuration	Page 2
04	IDE Setup: Visual Studio Code & PyCharm Integration	Page 2
05	The 3 Local Areas & Basic Commands Workflow	Page 3
06	Hands-on Real-Time Scenario: Building an Online Store	Page 3
07	Git Branching, Switching & Merging Strategies	Page 4
08	In-Depth Guide to Git Rebase (Standard & Interactive)	Page 4
09	Merge Conflict	Page 5

	Resolution & Git Stash	
10	Undoing Changes (Reset vs Revert) & Cherry-Picking	Page 5
11	Tricky Interview Questions & Real-Time Scenario Practice	Page 6
12	Advanced Engineering Scenarios & Enterprise Problem-Solving	Page 8

1. What is Git? (Beginner Analogies & Core Concepts)

Git is a distributed version control system (DVCS) that manages project source code over time. It keeps track of every single file modification, enabling developers to collaborate without overwriting code and quickly roll back when bugs occur.

Real-Time Analogy: Video Game Checkpoints

Imagine playing an adventure game. Before entering a difficult level, you save your progress at a checkpoint. If your character fails, you reload from that checkpoint! In Git, each **Commit** is a saved checkpoint in your project's history.

Why do we need Git?

- **Easy Rollback:** If a newly added feature breaks production, revert immediately to an older stable checkpoint.
- **Detailed Audit Trail:** Logs exactly who edited a file, when it occurred, and why (commit message).
- **Distributed & Fast:** Local working copy contains the complete history, making commands run instantly offline.
- **Seamless Collaboration:** Multiple developers can work concurrently on separate features without conflict.

2. Getting a Git Server & Creating a Remote Repository

A Git Server stores your central codebase in the cloud so team members can share and sync updates.

Popular Git Cloud Hosting Providers

- **GitHub:** Most widely used cloud host for open-source and enterprise projects.
- **GitLab:** Preferred for enterprise setups with built-in CI/CD pipelines.
- **Bitbucket:** Integrates closely with Jira and Atlassian toolchains.
- **AWS CodeCommit:** Fully managed cloud Git service hosted inside Amazon Web Services.

Steps to Create a Repository on GitHub

1. Sign in to `github.com`.
2. Click **New Repository**.
3. Enter a **Repository Name** (e.g., `my-app`).

4. Select **Public** or **Private**.
5. Check **Add a README file** and click **Create Repository**.

3. Windows Setup: Git Bash Installation & Configuration



Windows Setup (Git Bash):

Windows CMD/PowerShell lacks many Linux-style commands. **Git Bash** gives Windows users an emulation layer with native Linux commands (`ls`, `cd`, `mkdir`, `rm`) plus the Git CLI tools preconfigured out-of-the-box!

Steps to Install Git Bash on Windows:

1. Download the official Windows installer from `git-scm.com/downloads`.
2. Run the downloaded executable file (`Git-xx-64-bit.exe`).
3. Keep default selections, specifically ensuring:
 - **Select Components:** Check *Git Bash Here* and *Git GUI Here*.
 - **Default Editor:** Choose Nano, VS Code, or Vim.
 - **Adjusting PATH environment:** Select *Git from the command line and also from 3rd-party software*.
4. Complete installation. Right-click any folder in Windows File Explorer and select **"Git Bash Here"**.

Configuring Global Credentials (In Git Bash or Terminal):

```
# Set your global username and email address
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Verify global configuration
git config --list
```

4. IDE Setup: Visual Studio Code & PyCharm Integration

4.1 VS Code Integration

1. Open VS Code → **File** → **Open Folder** → Select your cloned repository folder.
2. Click **Source Control** icon on the left sidebar (`Ctrl + Shift + G`).
3. Stage modified files with the **+** button (`git add`) and write your commit message.
4. **Configuring Git Executable Path in VS Code (Windows):**
 - Go to **Settings** (`Ctrl + ,`) → Search `git.path`.
 - Click *Edit in settings.json* and add: `"git.path": "C:\\Program Files\\Git\\cmd\\git.exe"`
5. To set **Git Bash as VS Code Default Terminal**: Press `Ctrl + ~` to open terminal → Click dropdown arrow next to **+** → Select *Git Bash*.

4.2 PyCharm Integration

1. Open PyCharm → **Settings** (or **Preferences** on Mac) → **Version Control** → **Git**.
2. Verify *Path to Git executable* (On Windows: `C:\\Program Files\\Git\\cmd\\git.exe`). Click **Test**.
3. Open project → PyCharm detects Git automatically.
4. Press `Ctrl + K` (or `Cmd + K`) to open the Git Commit panel, write messages, and select **Commit and Push...**

5. The 3 Local Areas & Basic Commands Workflow

Git separates changes across three distinct local states before uploading to remote servers:

```
[ Working Area ]  --(git add)-->  [ Staging Area ]  --(git commit)-->  [ Local Repo ]  --(git
push)-->  [ Remote Repo ]
```

💡 Real-Time Analogy: Online Shopping Cart

- **Working Area:** Browsing items in the online store (editing files).
- **Staging Area:** Placing chosen items in your cart (`git add`). Nothing bought yet!
- **Local Repository:** Clicking "Place Order" (`git commit`). Transaction recorded locally.
- **Remote Repository:** Shipment arrives at your house (`git push`). Shared globally.

Essential Commands

```
# Check status of modified files
git status

# Stage files for commit
git add file.txt
git add .           # Stage all files

# Commit staged changes
git commit -m "Implement user authentication module"

# Push commits to remote master branch
git push origin master
```

6. Hands-on Real-Time Scenario: Building an Online Store

Follow this real-time example of initializing, editing, committing, and linking a project to GitHub.

```
# Step 1: Create project folder and open Git Bash inside it
mkdir store-app
cd store-app

# Step 2: Initialize Git local repository
git init

# Step 3: Create initial web page
echo "<h1>Welcome to My E-Commerce Store</h1>" > index.html

# Step 4: Check status, stage and commit
git status
git add index.html
git commit -m "Initial commit: Add homepage template"

# Step 5: Connect local repo to GitHub & push (Replace placeholders with your real info)
git remote add origin https://github.com/<your-username>/<your-repo-name>.git
git push -u origin master
```

7. Git Branching, Switching & Merging Strategies

A **Branch** is an isolated line of development. You can safely build new features without breaking stable code on the main branch.

Branch Command Reference

```
# Create a branch
git branch feature-cart

# Switch branch
git checkout feature-cart

# Shortcut: Create AND switch
git checkout -b feature-cart

# List branches
git branch -a

# Delete branch after merging
git branch -d feature-cart
git branch -D feature-cart    # Force delete unmerged branch
```

8. In-Depth Guide to Git Rebase (Standard & Interactive)

Rebasing is the process of moving or combining a sequence of commits to a new base commit. Instead of creating a cluttering merge commit, rebasing rewrites history to create a clean, linear commit log.

💡 Real-Time Analogy: Renovating a Multi-Story Building

Imagine building a feature on the 2nd floor of a house. Meanwhile, your teammates added foundational work to the 1st floor on `master`. Rebasing takes your entire 2nd-floor feature work, unplugs it, lifts it up, and attaches it directly on top of the newly updated 1st floor!

8.1 Standard Rebase Workflow

```
# Step 1: Switch to your feature branch
git checkout feature-payment

# Step 2: Rebase feature commits onto master
git rebase master

# Step 3: Switch back to master and fast-forward merge
git checkout master
git merge feature-payment
```

8.2 Interactive Rebase (`git rebase -i`)

Interactive rebase allows you to clean up local messy commits before submitting a Pull Request (PR):

```
# Interactively review and clean up the last 3 local commits
git rebase -i HEAD~3
```

Common Interactive Commands:

- `pick`: Keep the commit as is.
- `squash` (**s**): Combine a commit into the previous commit (merges 3-4 small commits into 1 clean commit).
- `reword` (**r**): Keep commit changes but edit the commit message.
- `drop` (**d**): Delete the commit completely.

⚠️ **The Golden Rule of Rebasing:** Never rebase commits that have already been pushed to a public/shared remote branch! Rebasing rewrites commit hashes, which will break the history for all other developers working on that shared branch.

9. Merge Conflict Resolution & Git Stash

9.1 Resolving Merge Conflicts

Conflicts occur when two developers edit the same file line differently. Git inserts markers inside the file:

```
<<<<<< HEAD (Current Change on Master)
title = "Welcome to Our Store"
=====
title = "Welcome to Super Store!"
>>>>>> feature-cart (Incoming Change)
```

To resolve: Open the file, remove conflict markers (`<<<`, `===`, `>>>`), keep the correct code, then run `git add <file>` and `git commit`.

9.2 Git Stash

Use `stash` when you need to switch tasks without committing incomplete work.

```
# Shelve current uncommitted work
git stash

# Switch branch & fix bug
git checkout master
# ... fix bug ...

# Restore stashed work
git checkout feature-cart
git stash pop
```

10. Undoing Changes (Reset vs Revert) & Cherry-Picking

10.1 Reset vs Revert

- `git reset`: Removes commits from local history. (*Use ONLY on unpushed local commits!*)

```
git reset --soft HEAD~1    # Keeps changes staged
git reset --hard HEAD~1    # Completely discards changes
```

- `git revert`: Creates a brand new commit that undoes changes of a previous commit. (*Safe for pushed commits!*)

```
git revert <commit-hash-id>
```

10.2 Git Cherry-Pick

Applies a specific commit from another branch to your active branch.

```
git cherry-pick <commit-hash-id>
```

11. Tricky Interview Questions & Real-Time Scenario Practice

TRICKY Q1

Why should Windows developers use Git Bash instead of Command Prompt?

Answer: Git Bash provides an authentic Unix/Linux shell environment on Windows. It allows developers to execute native Linux terminal utilities (`ls`, `grep`, `ssh`, `mkdir`, `find`) seamlessly alongside Git CLI tools without needing a full Linux Virtual Machine.

TRICKY Q2

What is the exact difference between ``git fetch`` and ``git pull``?

Answer: `git fetch` contacts the remote server and downloads new metadata/commits into isolated tracking branches (`origin/master`) without altering your local working directory. `git pull` is a combined macro: it executes `git fetch` AND immediately triggers a `git merge` into your active branch.

TRICKY Q3 (REAL-TIME SCENARIO)

Scenario: You accidentally deleted a critical feature branch locally using ``git branch -D feature-login`` before pushing it to remote. Is your code permanently lost? How do you recover it?

Answer & Real-Time Solution: No, it is not lost! Git retains dangling commits in its local reflog database for up to 30–90 days.

Recovery Steps:

```
# Step 1: Open reflog to view complete history of HEAD movements
git reflog

# Output snippet: 7f3a92b HEAD@{1}: commit: Add OAuth login feature

# Step 2: Re-create the lost branch pointing directly to that commit hash ID!
git checkout -b feature-login-recovered 7f3a92b
```

TRICKY Q4 (REAL-TIME SCENARIO)

Scenario: You ran ``git reset --hard HEAD~1`` on your local branch and realized you lost important code changes from that commit. How do you recover those lost changes?

Answer & Real-Time Solution: Even after a hard reset, the commit snapshot still exists in the local object store!

Recovery Steps:

```
# Step 1: Search reflog for the commit hash right BEFORE the hard reset occurred
git reflog

# Step 2: Use git cherry-pick or reset back to that commit hash
git reset --hard <commit-hash-before-reset>
```

TRICKY Q5 (REAL-TIME SCENARIO)

Scenario: You committed a database secret (``db_password = "supersecret"``) locally by mistake. How do you remove it properly? What if it was already pushed to GitHub?

Answer & Real-Time Solution:

- **If Unpushed:** Run `git reset --soft HEAD~1`, remove the secret from code, list the secret file in `.gitignore`, then re-commit cleanly.
- **If Already Pushed:** Revoke/rotate the database password **immediately** in production! Then use *BFG Repo-Cleaner* or `git filter-repo` to purge the credential from complete Git commit history before force-pushing.

TRICKY Q6

What is a Git Detached HEAD state, and how do you resolve it?

Answer: A "Detached HEAD" occurs when you check out a specific commit hash ID (`git checkout <commit-id>`) or remote tag instead of a named local branch. Any new commits made in this state are detached and not linked to any branch.

Resolution: To keep commits made in detached HEAD state, simply create a branch from it: `git switch -c new-feature-branch`.

TRICKY Q7 (REAL-TIME SCENARIO)

Scenario: During an interactive rebase (`git rebase -i`), a conflict occurs halfway through. How do you safely continue or cancel the rebase operation?

Answer & Real-Time Solution: When rebase pauses due to a conflict:

```
# Step 1: Open conflicted file, resolve markers, and stage resolved file
git add <file-name>

# Step 2: Continue the rebase (DO NOT run git commit!)
git rebase --continue

# Emergency Fallback: If things get messy and you want to abort safely:
git rebase --abort
```

TRICKY Q8 (REAL-TIME SCENARIO)

Scenario: You made 5 tiny commits like "fix typo", "update variable", "bug fix". How do you combine them into 1 meaningful commit before pushing to remote?

Answer & Real-Time Solution: Use interactive squashing:

```
# Step 1: Interactively review last 5 commits
git rebase -i HEAD~5

# Step 2: In editor, leave the first commit as 'pick', change 'pick' to 'squash' (or 's') for the remaining 4 commits
# Step 3: Enter a single clean commit message in the popup editor.
```

12. Advanced Engineering Scenarios & Enterprise Problem-Solving

TRICKY Q9 (DEBUGGING MASTERCLASS)

Scenario: A regression bug appeared in production out of 500 recent commits. How can `git bisect` automatically locate the exact culprit commit in seconds?

Answer & Real-Time Solution: `git bisect` uses a binary search algorithm to narrow down bad commits $\log(N)$ time.

```
# Step 1: Start bisecting
git bisect start
git bisect bad           # Current HEAD is broken
git bisect good v1.0.0   # Last known working release tag

# Step 2: Git checks out middle commit automatically. Test and report:
git bisect good          # or 'git bisect bad'

# Automated Execution with Test Script:
git bisect run pytest tests/test_payment.py
```

TRICKY Q10 (MULTITASKING SCENARIO)

Scenario: You are in the middle of a major architectural refactor with 30 modified files. Suddenly, an urgent P0 production hotfix is requested on `main`. `git stash` feels too messy for 30 files. What is `git worktree` and how does it solve this instantly?

Answer & Real-Time Solution: `git worktree` allows you to check out multiple branches simultaneously into separate physical directories on your file system from a single `.git` repo!

```
# Create a parallel workspace for hotfix in a separate directory
git worktree add ../hotfix-checkout main

# Open ../hotfix-checkout in PyCharm/VS Code, fix bug, commit, and push!
# Then return to your main workspace without ever touching your 30 uncommitted files.
```

TRICKY Q11 (REPO MAINTENANCE)

Scenario: Someone mistakenly committed a 600MB video file 3 months ago. Even though they ran `git rm video.mp4` in a later commit, cloning the repo takes 15 minutes because of that historical file. How do you permanently purge it from Git history?

Answer & Real-Time Solution: Running `git rm` only removes the file from future snapshots—the 600MB object remains stored in historical commit objects!

```
# Use official recommended tool: git-filter-repo
git filter-repo --path video.mp4 --invert-paths

# Force prune dangling blobs and reclaim space
git reflog expire --expire=now --all
git gc --prune=now --aggressive
```

TRICKY Q12 (TEAM WORKFLOW & SAFETY)

Scenario: You rebased your feature branch locally. Why is `git push --force-with-lease` mandatory instead of `git push --force` when updating your remote Pull Request branch?

Answer & Real-Time Solution: `git push --force` blindly overwrites the remote branch regardless of what happened on the server. If a teammate pushed a commit to your PR branch while you were rebasing, `--force` will destroy their work forever! `--force-with-lease` checks if the remote ref has received un-fetched commits—if so, it aborts the push, preventing accidental overwrites.

TRICKY Q13 (AUTOMATION TRICK)

What is `git rerere` (Reuse Recorded Resolution) and how does it save hours during long feature development?

Answer & Real-Time Solution: When working on long-lived branches, you may rebase or merge `main` repeatedly and resolve the exact same merge conflict multiple times.

```
# Enable rerere globally
git config --global rerere.enabled true
```

When enabled, Git records how you resolved a conflict the first time. The next time the same conflict appears in any merge/rebase, Git automatically applies your previous resolution instantly!

TRICKY Q14 (HISTORY TRACING)

Scenario: A function bug was traced to line 85 of `auth_service.py`. How do you trace who edited that specific line 2 years ago, even if the file was renamed 3 times?

Answer & Real-Time Solution: Use `git blame` with line range (`-L`) and copy/rename detection (`-C` / `-M`):

```
# Blame specific lines across file renames
git blame -C -L 80,90 auth_service.py

# Trace line evolution through history
git log -L 85,85:auth_service.py
```

TRICKY Q15 (SECURITY & TRUST)

Why is `git config user.email` insecure on its own in enterprise environments, and how do SSH/GPG signed commits protect production builds?

Answer & Real-Time Solution: Anyone can set `git config user.email "ceo@company.com"` and fake a commit under anyone else's name! To guarantee authenticity, enterprises require signed commits:

```
# Configure SSH key signing in Git
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub

# Make a cryptographically signed commit
git commit -S -m "Verified release commit"
```

GitHub/GitLab verifies your public key and displays a green **"Verified"** badge next to the commit.